

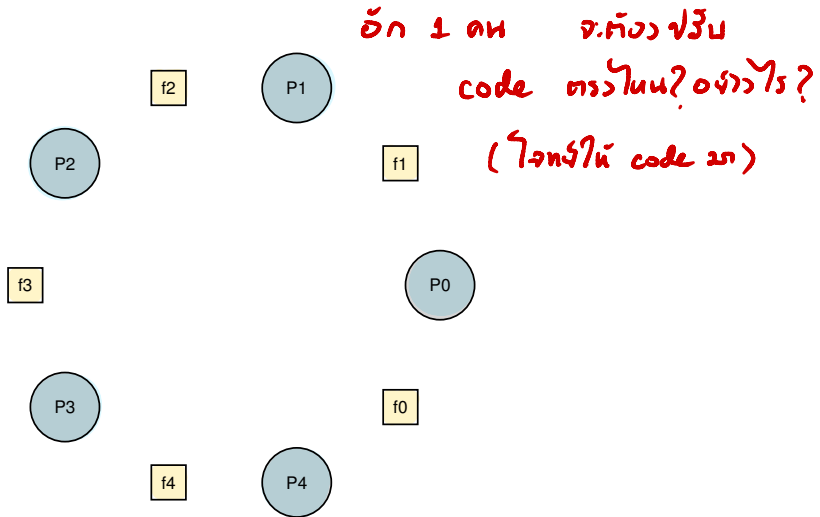


Figure 31.14: The Dining Philosophers

Here is the basic loop of each philosopher:

```
while (1) {
    think();
    getforks();
    eat();
    putforks();
}
```

The key challenge, then, is to write the routines `getforks()` and `putforks()` such that there is no deadlock, no philosopher starves and never gets to eat, and concurrency is high (i.e., as many philosophers can eat at the same time as possible).

Following Downey's solutions [D08], we'll use a few helper functions to get us towards a solution. They are:

```
int left(int p) { return p; }
int right(int p) { return (p + 1) % 5; }
```

When philosopher p wishes to refer to the fork on their left, they simply call `left(p)`. Similarly, the fork on the right of a philosopher p is referred to by calling `right(p)`; the modulo operator therein handles the one case where the last philosopher ($p=4$) tries to grab the fork on their right, which is fork 0.

We'll also need some semaphores to solve this problem. Let us assume we have five, one for each fork: `sem_t forks[5]`.

1. (15 Min) The following code attempts to implement a locking mechanism for two threads using two shared flags declared as:

```
int flag[2];
```

which are both initialized to 0. Both threads continuously execute a loop which contains a critical section of code.

Thread 0

```
while (1) {  
    while (flag[1] == 1); // spin  
    flag[0] = 1;  
    // start critical section  
    ... ;  
    // end critical section  
    flag[0] = 0;  
    ... ;  
};
```

Thread 1

```
while (1) {  
    while (flag[0] == 1); // spin  
    flag[1] = 1;  
    // start critical section  
    ... ;  
    // end critical section  
    flag[1] = 0;  
    ... ;  
};
```

Does this code satisfy the evaluation criteria? If not, which of the requirements are not met? Explain your answer.

This code does not satisfy the evaluation criteria

- **Correctness** : There is the case where both threads set the flag to 1 and both threads are thus able to enter the critical section, as you can see in the table below

Thread 0	Thread 1
<pre>while (flag[1] == 1) interrupt: switch to Thread 1 flag[0] = 1; //set flag to 1 (too!)</pre>	<pre>while (flag[0] == 1) flag[1] = 1; interrupt: switch to Thread 0</pre>

As you can see from this interleaving, this code failed to provide mutual exclusion.

- **Performance** : Thread waits to acquire a lock that is already held: it endlessly checks the value of flag, known as spin-waiting. Spin-waiting wastes time waiting for another thread to release a lock. The waste is exceptionally high on uniprocessor, where the thread that the waiter is waiting for cannot even run.
- **Fairness** : The code doesn't provide any fairness guarantees that a waiting thread will ever enter the critical section. A thread spinning may spin forever and may lead to starvation.

2. (15 Min) There are three resource types, R, S, and T, each with a single instance and there are 2 processes P1 and P2. Use Resource-Allocation Graphs to model the following situations. In each case, **determine** if there is a deadlock. You need **only show** the final resource allocation graph for each case in your answer.

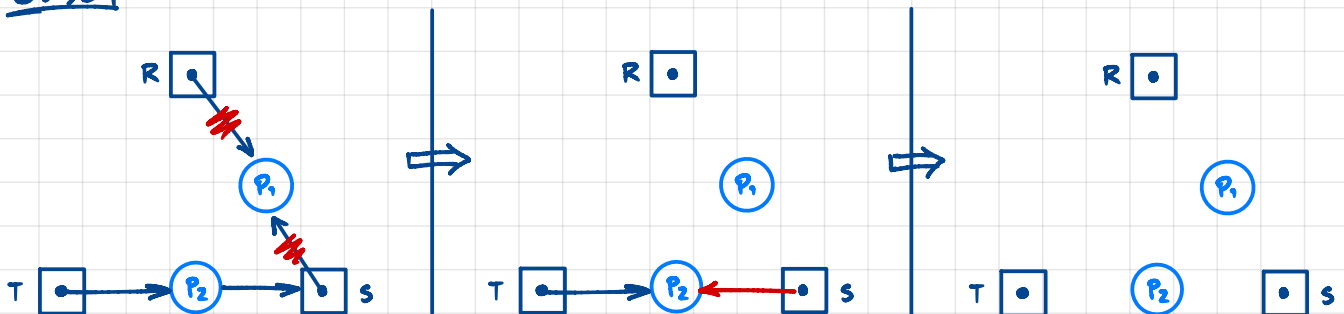
Case 1

P1 requests R
P2 requests T
P1 requests S
P2 requests S
P1 releases R
P1 releases S

Case 2

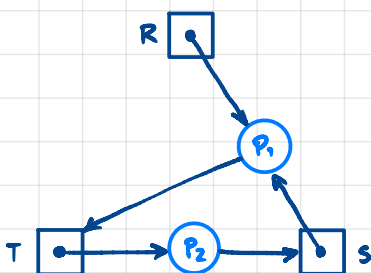
P1 requests R
P2 requests T
P1 requests S
P2 requests S
P1 requests T

Case 1



∴ There is no deadlock, every process can execute until completion.

Case 2



∴ Deadlock arises because four conditions happen simultaneously

- 1) Mutual Exclusion
- 2) No Preemption
- 3) Hold and Wait

- P1 is holding R, S and waiting for T
- P2 is holding T and waiting for S

- 4) Circular Wait

P1 is waiting for a resource that is held by P2 and P2 is waiting for a resource that is held by P1.

Part2-1

1. What is the benefit of thread over process when context switching occurs?
2. Explain how can a race condition effect the result of concurrency execution?
3. Give an example of code to create a lock from test_and_set()
4. How can you eliminate the time wasted in spinning loop in lock?

1) Threads share the same address space and thus can access the same data. There is no need to switch which page table we are using. As a result, The context switch among threads is faster and consume resources efficiently.

2) Race condition make the results depend on the timing execution of the code. With some bad luck (i.e, context switches that occur at untimely points in the execution), we get the wrong result. In fact, we may get a different result each time.

```
3) int test_and_set(int *old_ptr, int new) {
    int old = *old_ptr;
    *old_ptr = new;
    return old;
}
```

```
void lock(lock_t *lock) {
    while (test_and_set(&lock->flag, 1) == 1) ;
}
```

```
void unlock(lock_t *lock) {
    lock->flag = 0;
}
```

4) When you are going to spin, instead give up the CPU to another thread by using an operating system primitive "yield()". yield is simply a system call that moves the caller from the running state to the ready state, and thus promotes another thread to running. However there is potential for waste and no prevention of starvation.

Thus we can use OS support, as well as a queue to keep track of which Threads are waiting to acquire the lock;

- park() to put a calling thread to sleep
- unpark(threadID) to wake a particular thread as designated by threadID

CPE 326 Operating System Final Exam

Part 2-2

How can we prevent deadlock in dining philosopher problem? Explain in detail.

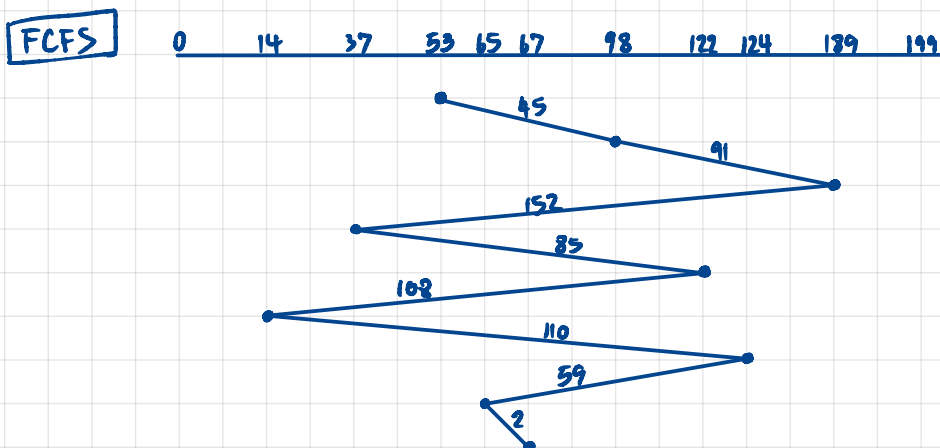
The simplest way is to change how forks are acquired by at least one of the philosophers. Specifically, let's assume that philosopher 4 (the highest number one) acquires the forks in a different order (grabs right before left while others grab left before right). Thus, there is no situation where each philosopher grabs one fork and is stuck waiting for another; the cycle of waiting is broken.

CPE326 Operating System Final Exam 2022

Part2-3

If a disk has cylinder 0-199, and a head starts at position 53. The following positions are in reading queue {98,189,37,122,14,124,65,67}. Illustrate the seek path and calculate the total head movement for the following algorithms:

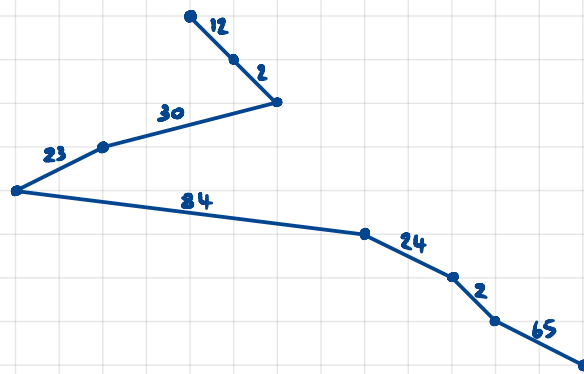
1. FCFS
2. SSTF
3. C-SCAN (left to right)
4. C-LOOK (left to right)



Total head movement
= 652

SSTF

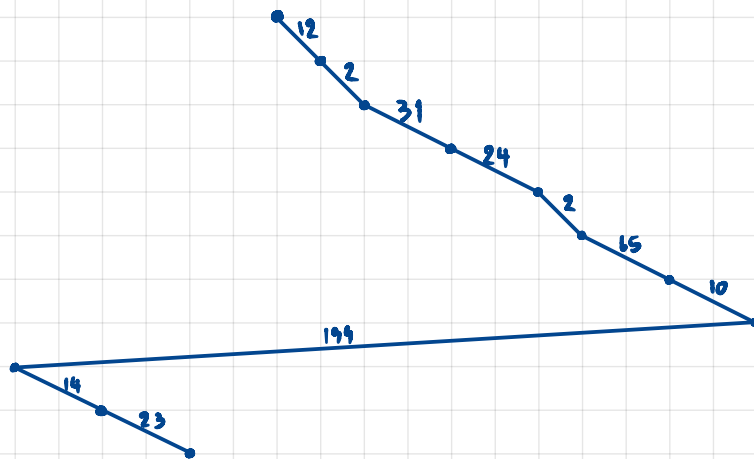
0 14 37 53 65 67 98 122 124 139 199



Total head movement
= 242

C-SCAN

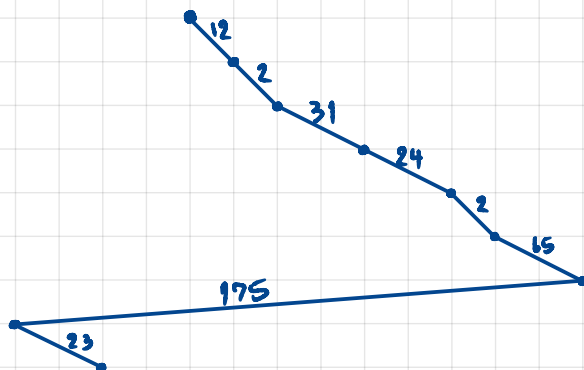
0 14 37 53 65 67 98 122 124 139 199



Total head movement
= 382

C-LOOK

0 14 37 53 65 67 98 122 124 139 199



Total head movement
= 334

3. There are three resource types, R, S, and T, each with a single instance and there are 2 processes P1 and P2. Use resource-allocation graphs to model the following situations. In each case, determine if there is a deadlock. You need only show the final resource allocation graph for each case in your answer.

Case 1

P1 requests R
P2 requests T
P1 requests S
P2 requests S
P1 releases R
P1 releases S

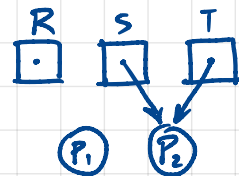
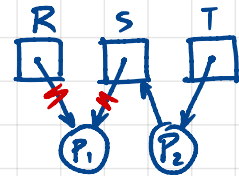
Case 2

P1 requests R
P2 requests T
P1 requests S
P2 requests S
P2 requests R
P1 releases R
P1 releases S

Case 3

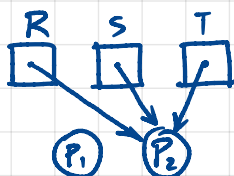
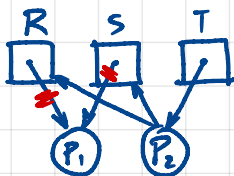
P1 requests R
P2 requests T
P1 requests S
P2 requests S
P1 requests T

Case 1



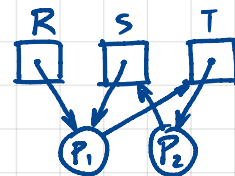
No deadlock

Case 2



No deadlock

Case 3



Deadlock!

∴ Mutual exclusion, No Preemption, Hold and wait and Circular wait